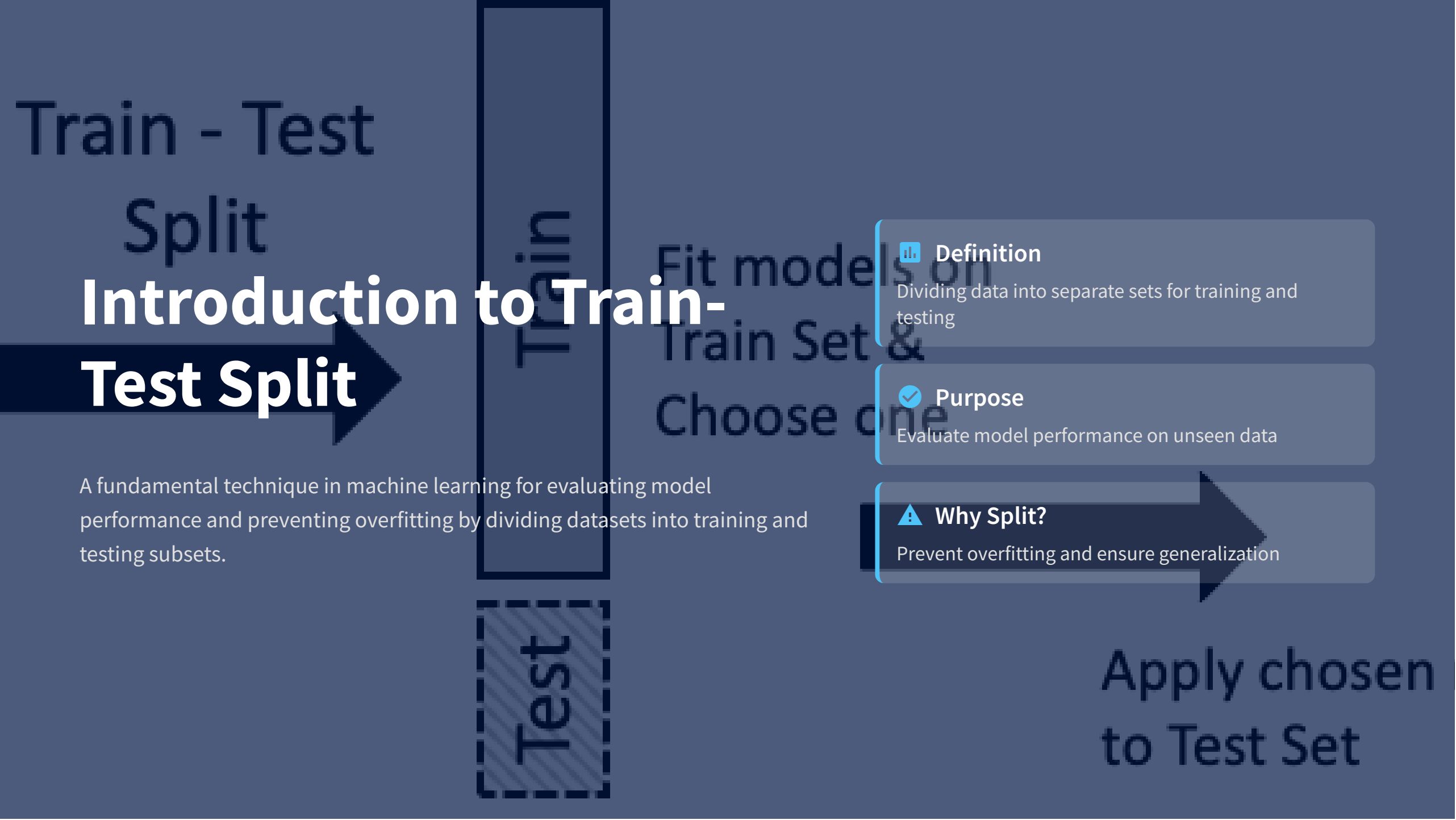



# Introduction to Train-Test Split

A fundamental technique in machine learning for evaluating model performance and preventing overfitting by dividing datasets into training and testing subsets.

**Definition**

Dividing data into separate sets for training and testing

**Purpose**

Evaluate model performance on unseen data

**Why Split?**

Prevent overfitting and ensure generalization

# Train-Test Split Procedure

1

## Arrange Data

Separate dataset into features (X) and target (y)

2

## Split Data

Divide data into training and testing sets

3

## Train Model

Fit the model on training data (X_train, y_train)

4

## Test Model

Evaluate performance on unseen test data (X_test, y_test)

## Common Split Ratios

**80/20** - Most common

**70/30** - Good balance

**67/33** - Alternative

# Methods for Splitting Data

## Random Splitting

Randomly shuffles data and splits into training and testing sets based on specified percentages.

**Best for:**

- Large datasets
- General machine learning tasks
- When class distribution is balanced

## Stratified Splitting

Divides data while preserving the proportion of classes or categories in both sets.

**Best for:**

- Imbalanced datasets
- Classification problems
- When minority classes are important

## Time-Based Splitting

Organizes data by time, ensuring past data is in training set and future data is in testing set.

**Best for:**

- Time series data
- Financial forecasting
- When temporal patterns matter

## Overfitting

Model learns training data too well, including noise and random fluctuations

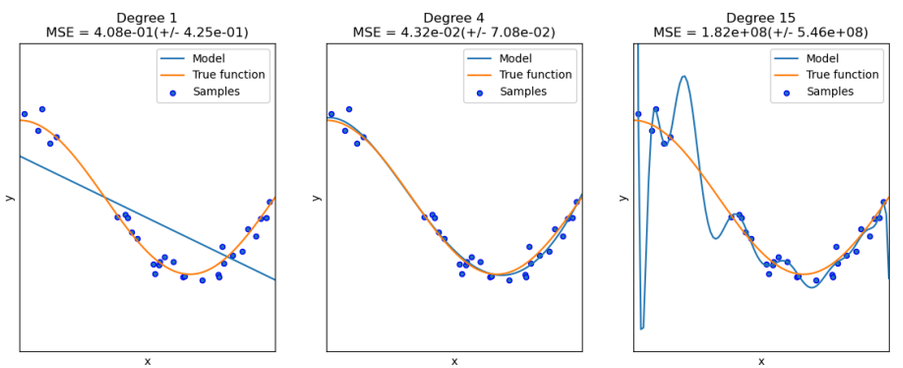
- **High training accuracy** but poor test performance
- **Complex models** with too many parameters
- **Memorization** instead of learning patterns

## Poor Generalization

Model fails to perform well on new, unseen data

- **Unreliable predictions** in real-world scenarios
- **Inconsistent performance** across different datasets
- **False confidence** in model capabilities

### Overfitting vs. Proper Fitting



# Python Implementation for Train-Test Split

## Code Example with scikit-learn

```
# Import necessary libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier

# Load dataset
df = pd.read_csv('data.csv')

# Separate features and target
X = df.drop('target', axis=1)
y = df['target']

# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.25,     # 25% for testing
    random_state=42,    # For reproducibility
    stratify=y          # Preserve class distribution
)

# Initialize and train the model
model = RandomForestClassifier(n_estimators=100)
model.fit(X_train, y_train)

# Evaluate the model
train_score = model.score(X_train, y_train)
test_score = model.score(X_test, y_test)

print(f"Training accuracy: {train_score:.4f}")
print(f"Test accuracy: {test_score:.4f}")
```

## Key Parameters

- **test_size**: Proportion of data for testing (0.2-0.3)
- **random_state**: Ensures reproducible splits
- **stratify**: Maintains class distribution
- **shuffle**: Randomizes data before splitting

## Expected Output

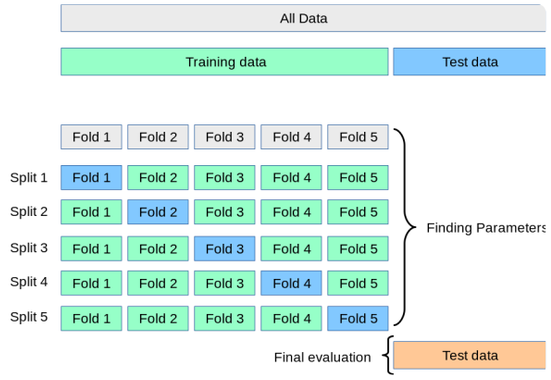
```
Training accuracy: 0.9876 Test accuracy: 0.8543 X_train shape: (1200, 10) X_test shape: (400, 10) y_train shape: (1200,) y_test shape: (400,)
```

## Definition & Purpose

A resampling technique to evaluate ML models on limited data samples by using different portions of data for training and testing across multiple iterations.

- **More reliable** performance estimation
- **Better utilization** of available data
- **Reduces risk** of overfitting
- **Ensures model** generalizes well to unseen data

### Cross-Validation Process




## Cross-Validation vs. Train-Test Split

| Aspect | Train-Test Split | Cross-Validation |
|---|---|---|
| **Data Usage** | Single split | Multiple splits |
| **Training** | Once | Multiple times |
| **Evaluation** | Single score | Average of scores |
| **Reliability** | Depends on split | More stable |
| **Computation** | Faster | More expensive |

# Types of Cross-Validation

## Holdout Validation

Simplest method where data is split into training and testing sets once, typically using 50-50 split.

### Key Features

- **Quick & simple** to implement
- Only uses **50% of data** for training
- Results can vary significantly based on split
- Higher bias compared to other methods

## LOOCV (Leave One Out)

Model is trained on all data except one observation, which is used for testing. Process repeats for each data point.

### Key Features

- **Maximum data usage** for training
- Low bias but potentially high variance
- Computationally **very expensive** for large datasets
- Unbiased performance estimate

## Stratified Cross-Validation

Ensures each fold maintains the same class distribution as the full dataset, especially useful for imbalanced datasets.

### Key Features

- **Preserves class ratios** in each fold
- Ideal for **classification problems**
- Prevents model bias toward majority class
- More reliable performance for imbalanced data

## K-Fold Cross-Validation

Dataset is divided into k equal-sized folds. Model is trained on k-1 folds and tested on the remaining fold, repeated k times.

### Key Features

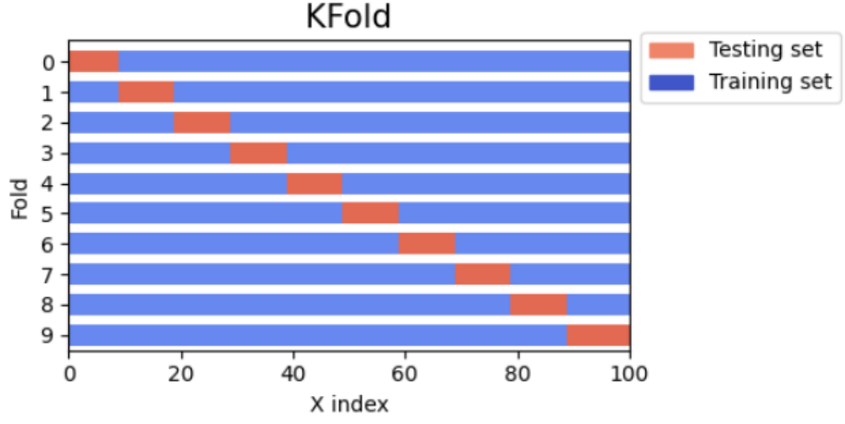
- **Balance between** bias and variance
- Commonly uses k=5 or k=10
- Each data point used for both training and testing
- More reliable than single train-test split

## How It Works

Dataset is divided into **k equal-sized folds**. Model is trained on k-1 folds and tested on the remaining fold, repeated k times.

- Each data point is used for **both training and testing**
- Performance is averaged across all k iterations
- Typical values: k=5 or k=10
- Balances bias-variance tradeoff

### K-Fold Cross-Validation Process




## Example with k=5

| Iteration | Fold 1 | Fold 2 | Fold 3 | Fold 4 | Fold 5 |
|---|---|---|---|---|---|
| 1 | Test | Train | Train | Train | Train |
| 2 | Train | Test | Train | Train | Train |
| 3 | Train | Train | Test | Train | Train |
| 4 | Train | Train | Train | Test | Train |
| 5 | Train | Train | Train | Train | Test |

1. Split data into 5 folds
2. Train on 4 folds
3. Test on 1 fold
4. Repeat 5 times
5. Average results

# Comparison: Train-Test Split vs Cross-Validation

## Train-Test Split

### Advantages

- **Fast** and computationally efficient
- **Simple** to implement
- Works well with **large datasets**
- Clear separation of training and testing

### Disadvantages

- Results depend on **specific split**
- May **waste data** for testing
- Less reliable performance estimate

VS

## Cross-Validation

### Advantages

- **More reliable** performance estimate
- **Better utilization** of data
- Reduces risk of **overfitting**
- Works well with **small datasets**

### Disadvantages

- **Computationally expensive**
- More **complex** to implement
- May not be suitable for **very large datasets**

## When to Use Which Method

| | |
|---|---|
| **Large Datasets** | Train-Test Split (faster, sufficient data for testing) |
| **Small/Medium Datasets** | Cross-Validation (better data utilization) |
| **Quick Prototyping** | Train-Test Split (faster iteration) |

# Python Implementation for Cross-Validation

## K-Fold Cross-Validation with scikit-learn

```
# Import necessary libraries
import pandas as pd
import numpy as np
from sklearn.model_selection import cross_val_score, KFold
from sklearn.ensemble import RandomForestClassifier

# Load dataset
df = pd.read_csv('data.csv')

# Separate features and target
X = df.drop('target', axis=1)
y = df['target']

# Initialize model
model = RandomForestClassifier(n_estimators=100)

# Define K-Fold cross-validation
k = 5  # Number of folds
kf = KFold(n_splits=k, shuffle=True, random_state=42)

# Perform cross-validation
cv_scores = cross_val_score(model, X, y, cv=kf)

# Display results
print(f"Cross-validation scores: {cv_scores}")
print(f"Mean CV score: {cv_scores.mean():.4f}")
print(f"Standard deviation: {cv_scores.std():.4f}")
```

## Key Components

- **KFold**: Creates k-fold iterator
- **cross_val_score**: Evaluates model using CV
- **n_splits**: Number of folds (k)
- **shuffle**: Randomizes data before splitting
- **random_state**: Ensures reproducibility

## Expected Output

```
Cross-validation scores: [0.846, 0.852, 0.848, 0.851, 0.849] Mean CV score: 0.8492 Standard deviation: 0.0023
```

Lower standard deviation indicates more **stable performance** across different data splits

# Best Practices and Considerations

## Choosing the Right Method

- **Large datasets**: Use train-test split for efficiency
- **Small datasets**: Use cross-validation for better data utilization
- **Imbalanced classes**: Use stratified cross-validation
- **Time series**: Use time-based splitting
- **Final model evaluation**: Use cross-validation for reliability

## Common Pitfalls to Avoid

- **Data leakage**: Preprocessing before splitting
- **Wrong k value**: Too low (high bias) or too high (high variance)
- **Ignoring class imbalance**: Not using stratified splitting
- **Random state**: Not setting for reproducibility
- **Multiple testing**: Using test set for hyperparameter tuning

**Always keep a separate test set for final model evaluation after cross-validation and hyperparameter tuning**